

RelaySaathi

Master Team Brief — Road Safety Hackathon 2026 (RoadSoS Track)

Organizer: CoERS & RBG Labs, IIT Madras

Deadline: May 31, 2026, 11:59 PM IST

Version: 1.0

Audience: Full hackathon team

The network failed. The golden hour didn't.

RoadSoS Track · IIT Madras

RelaySaathi Master Team Brief

 May 2026  Full team document  Deadline: May 31, 2026

The network failed. The golden hour didn't.

RelaySaathi — Master Team Brief

Project: RelaySaathi (Road Safety Hackathon 2026 — RoadSoS Track) **Organizer:** CoERS & RBG Labs, IIT Madras (with MoRTH) **Document version:** 1.0 **Last updated:** May 2026 **Audience:** Full hackathon team — developers, presenters, data engineers

Table of Contents

[Executive Summary](#)[Hackathon Alignment](#)[Product Vision & One-Line Pitch](#)[Why We Chose This Architecture](#)[What We Rejected \(And Why\)](#)[Core Innovation](#)[System Architecture](#)[End-to-End Pipeline](#)[User Flows & Mind Map](#)[START Triage FSM Specification](#)[Golden Hour Packet \(GHP\) Specification](#)[Data Pipeline & Database](#)[Offline & Spatial Routing](#)[AI Chatbot Architecture](#)[Human QR Relay System](#)[Tech Stack](#)

[Project Structure](#)[Implementation Plan \(4 Weeks\)](#)[Execution Checklist by Role](#)[Demo Day Script \(90 Seconds\)](#)[Submission Requirements](#)[Verification & Testing Plan](#)[Risks & Mitigations](#)[Competitive Advantages](#)[Glossary \(Quick Reference\)](#)[Open Decisions & Team Actions](#)[Contacts & Resources](#)

1. Executive Summary

RelaySaathi is an **offline-first, AI-powered emergency chatbot** (Progressive Web App) designed for the **RoadSoS** problem statement at the IIT Madras National Road Safety Hackathon 2026. When a road accident happens — often with **no mobile signal** on highways — victims and bystanders lose precious minutes trying to explain location, injury severity, and which hospital to use. RelaySaathi compresses that chaos into a **60-second guided triage conversation**, routes the victim to the **correct trauma-tier facility** (not just the nearest clinic), builds a **Golden Hour Packet (GHP)** dispatch brief, and if offline, passes that packet through a **human QR relay chain** until any phone regains signal and can SMS **108/112**. **Tagline:** *The network failed. The golden hour didn't.*

2. Hackathon Alignment

Event Details

Item	Detail
Theme	AI in Road Safety (2026 focus: AI-powered chatbots)
Our track	RoadSoS — location-based emergency services during accidents
Team size	1–10 members (Unstop enforced)
Submission	Unstop only — one submission, one problem statement
Indian track deadline	May 31, 2026, 11:59 PM IST (extended)
Stage 1	Online prototype + code + DB + 7 slides + Word doc
Stage 2	Shortlisted teams present live at IIT Madras

RoadSoS Evaluation Criteria (What Judges Score)

Criterion	How RelaySaathi Addresses It
Data reliability & geographic accuracy	Pre-built, verified SQLite POI database from OSM + manual corridor verification
Valid emergency contacts fetched & displayed	Hospitals, police, ambulance, towing — with real phone tags where available
Offline capability	FSM triage + sql.js local DB + Service Worker + corridor packs + QR relay
Innovation & extra features	GHP pre-dispatch brief + human sneakernet relay + trauma-tier routing + NH km markers
Cross-country integration	Country/state config packs (IN + BIMSTEC: NP, BD, LK, etc.)
AI chatbot requirement	Conversational UI with LLM slot-filling over deterministic START FSM
Open-source preference	OSM, OSRM, sql.js, Llama/Mistral (Groq/Ollama), no proprietary lock-in

Submission Deliverables Checklist

- ☐ Full source code (Python preferred for backend/ingest)
- ☐ **Structured database** (`emergency_seed.db` + export)
- ☐ **Exactly 7 slides** (Welcome + Thank You included)
- ☐ **Word document:** codebase overview, dependencies, assumptions
- ☐ Working Unstop submission before deadline

3. Product Vision & One-Line Pitch

Problem

India records ~1.7 lakh road deaths/year. On highways, accidents often occur in **dead zones** with no signal. Victims cannot:

- Explain exact location (lat/lng is meaningless to most users)
- Communicate injury severity clearly to 108
- Know whether the nearest facility is a **trauma center** or a small clinic
- Pass information to dispatch when their phone has no connectivity

Solution

RelaySaathi is a **conversational emergency agent** that:

Runs **START medical triage** (international first-responder protocol)

Finds the **right tier** of emergency facility offline

Generates a **dispatch-ready Golden Hour Packet**

Relays that packet via QR to a bystander when the victim has no signal

Pitches (Memorize)

Audience	Pitch
30 sec (judges)	RelaySaathi is an offline AI emergency chatbot. In 60 seconds it triages injuries, routes RED cases to trauma centers, builds a 108-ready brief, and if there's no signal — a bystander scans a QR and carries that brief until they get network.
Technical	LLM-as-skin, FSM-as-spine, sql.js POI store, Haversine spatial query with trauma-tier filter, lz-string GHP encoding, IndexedDB human relay.
Emotional	When the network fails, the golden hour doesn't.

4. Why We Chose This Architecture

We intentionally built a system that is:

Demo-proof — works in Chrome on two phones with airplane mode

Offline-first — core path never requires live API calls

Medically grounded — START triage protocol, not arbitrary questions

Clinically routed — trauma-tier matching, not nearest-pin routing

Information-centric — solves *dispatch delay*, not *sensor novelty*

Hackathon-aligned — AI chatbot is the product, not a sidebar feature

Open-source friendly — OSM, OSRM, open LLMs, SQLite

5. What We Rejected (And Why)

Early ideation explored a **sensor-heavy "smart vehicle mesh"** approach. After engineering verification, we **deliberately killed** those features. This section documents why — so the team stays aligned and does not re-introduce scope creep.

Rejected Feature Matrix

Rejected Idea	What It Tried To Do	Why We Killed It
Ultrasonic V2V	Phones communicate via 19–21 kHz sound between vehicles	Browsers block mic/audio without user gesture; road noise interference; iOS ultrasonic limits; unreliable demo
BLE / DTN mesh	Crash packets relay through passing cars via Bluetooth	Web Bluetooth cannot advertise/beacon; no BLE mesh in browsers; iOS has no Web Bluetooth
Wi-Fi Direct beacons	Broadcast distress via Wi-Fi Direct SSID	Not available in web browser APIs
Crash audio ML	Always-on mic detects tire screech/impact	False positives; privacy issues; battery drain; research-grade ML scope
Capacitor/native hybrid	Wrap PWA in native shell for "real" Bluetooth	Doubles build complexity; judges won't sideload APKs; contradicts accessible web prototype
Curvature/anxiety physics engine	Predict safe speed on curves from OSM geometry	Wrong problem statement (RoadWatch); OSM geometry too sparse; missing friction/banking data
Background volunteer GPS dispatch	Track volunteers when app minimized	PWAs cannot reliably background-track GPS (especially iOS); needs gov integration
Base64-only SMS coords	Encode GPS in SMS as Base64	108 operators cannot decode; useless to dispatchers
Always-on "Drive Mode"	Mic + wake lock + audio on launch	Scary UX; permission denial; battery drain

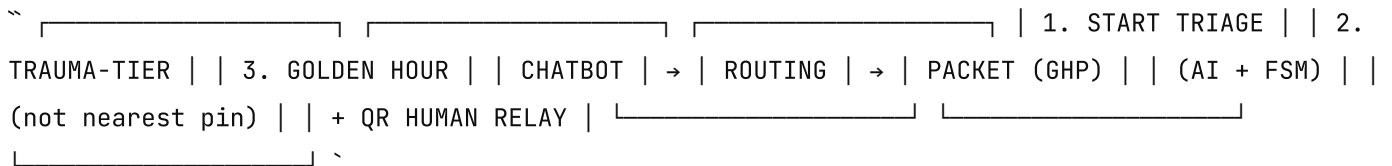
Strategic Lesson

Judges score emergency chatbots with valid POI data and offline reliability — not ultrasonic chirps between cars.

Our creative differentiator is **information surviving dead zones** (GHP + human QR relay), not **machines talking to machines**.

6. Core Innovation

The Three Pillars



Innovation #1 – Golden Hour Packet (GHP)

A **structured, dispatch-ready brief** – not raw chat logs:

- Triage color (RED/YELLOW/GREEN/BLACK)
- GPS + landmark + optional NH km marker
- Victim status flags
- Selected facility (name, type, phone, ETA, distance)
- State emergency number (108/112) + language
- Integrity hash (tamper detection)
- Relay chain audit log

Why it matters: 108 dispatchers lose minutes asking "where are you? what happened?" GHP **pre-answers** those questions.

Innovation #2 – Human QR Relay (Sneakernet)

When victim phone has **no signal**:

Triage completes offline → GHP generated

Victim screen shows **QR code** + human-readable brief

Bystander taps **[Scan Distress QR]** on their phone

Packet stored in **IndexedDB**

When online event fires → auto-compose **SMS to 108** with readable GHP text

Why it matters: Honest, demo-proof, works on every smartphone. No mesh fantasy.

Innovation #3 – Trauma-Tier Routing

Triage	Route To	trauma_tier
RED	Trauma center / surgical ER	1, 2
YELLOW	General hospital with ER	2, 3
GREEN	Clinic / minor care	3 (+ police if needed)
BLACK	Deceased after airway attempt	Police / 108 notification

Why it matters: Nearest hospital might be a PHC unable to handle head trauma. We route by **capability**, not distance alone.

7. System Architecture

High-Level Architecture Diagram

```
`mermaid flowchart TB
    subgraph Client["RelaySaathi PWA (Next.js 14)"]
        Gate[Landing Gate]
        Chat[Triage Chat UI]
        Agent[LLM Slot Filler]
        FSM[START Triage FSM]
        Router[Offline Spatial Router]
        GHP[GHP Builder + Compressor]
        QR[QR Generator / Scanner]
        IDB[(IndexedDB Relay Store)]
        SW[Service Worker / Serwist]
        SQL[(sql.js + emergency_seed.db)]
        Map[Leaflet Offline Map]
    end

    subgraph Build["Build-Time Pipeline"]
        Ingest[ingestCorridors.py]
        OSM[Overpass API / OpenStreetMap]
        DBFile[emergency_seed.db]
    end

    subgraph Online["Online Enhancements"]
        LLM[Groq / Ollama - Llama or Mistral]
        OSRM[OSRM Routing API]
    end

    Gate --> Chat
    Chat --> Agent
    Agent --> FSM
    FSM --> Router
    Router --> SQL
    SQL --> OSRM
    OSRM --> FSM
    FSM --> GHP
    GHP --> QR
    QR --> IDB
    IDB --> SW
    SW --> SQL
    SQL --> Ingest
    Ingest --> OSM
    OSM --> DBFile
    DBFile --> SQL
    SQL --> Agent
    Agent --> LLM
    LLM --> GHP
    GHP --> Map`
```

Layer Responsibilities

Layer	Responsibility	Must Work Offline?
UI (Next.js PWA)	Chat, map, QR, scan, landing gate	Yes (cached shell)

Layer	Responsibility	Must Work Offline?
FSM (TypeScript)	Medical triage logic	Yes – mandatory
LLM (optional)	Natural language → structured slots	No (enhancement only)
sql.js + SQLite	POI lookup	Yes – mandatory
GHP encoder	Serialize, compress, hash	Yes – mandatory
QR relay	Encode/decode, IndexedDB, SMS trigger	Yes – mandatory
Service Worker	Cache app shell, DB, wasm, tiles	Yes – mandatory
OSRM	Road-accurate ETA	No (fallback: Haversine ETA)
Ingest script	Build POI database	Build-time only

8. End-to-End Pipeline

Pipeline A – Build-Time (Developer Machine / CI)

` OpenStreetMap data ↓ Overpass API queries (hnr: hospitals, clinics, police, ambulance) ↓ ingestCorridors.py • Parse names, phones, types • Assign trauma_tier (1/2/3) • Tag corridor + state • Manual verification CSV merge (top 50 POIs per demo corridor) ↓ public/emergency_seed.db ↓ Bundled into PWA + submitted with hackathon deliverables `

Pipeline B – Runtime (Victim Path)

` User taps [Start Emergency Triage] ↓ Browser requests GPS (single getCurrentPosition) ↓ Chatbot conversation (LLM online / scripted offline) ↓ Slots fill START FSM states ↓ FSM outputs triage color ↓ Spatial query: bbox filter → Haversine sort → trauma-tier filter ↓ Select best facility + compute ETA ↓ Build GoldenHourPacket + SHA-256 integrity hash ↓
└──────────────────┘ | ONLINE | OFFLINE | • Call 108 / hospital | • Show GHP on screen | • SMS / WhatsApp share | • Display QR code | • Show map route | • Prompt bystander

scan _____ `

Pipeline C – Runtime (Bystander Relay Path)

` Bystander taps [Scan Distress QR] ↓ Camera decodes QR payload ↓ Deserialize + verify integrity hash ↓ Store in IndexedDB + append relayChain entry ↓ Listen for navigator 'online' event ↓ Format human-readable SMS template (Tamil/Hindi/English) ↓ Trigger sms:108?body=... OR navigator.share() OR copy-to-clipboard ↓ (Optional) POST anonymized GHP to backend audit endpoint `

9. User Flows & Mind Map

Landing Gate (3 Entry Points)

` _____ | RelaySaathi Home | _____
 _____ ▼ ▼ ▼ [Start Emergency] [Scan Distress QR] [Download Corridor Pack] (Victim / (Bystander) (Pre-trip offline prep) Bystander-help) `

Mind Map – Full System

` RelaySaathi | INPUT | | Voice (Web Speech API) | | Text chat | | GPS + optional NH km marker | PROCESSING | | AI layer (LLM slot filling – online) | | FSM layer (START triage – always) | | Routing layer (trauma-tier + distance) | | GHP builder (structured output) | DATA | | emergency_seed.db (SQLite via sql.js) | | IndexedDB (relay packets) | | Service Worker cache (shell, wasm, tiles) | OUTPUT | | Map with ranked facilities | | Human-readable GHP on screen | | QR code (compressed payload) | | Call / SMS / WhatsApp actions | | Bystander relay → 108 SMS | SUBMISSION | Source code | Database file | 7 slides | Word doc `

Flowchart – Triage Decision (Simplified)

```
mermaid flowchart TD
    Start([Start Emergency]) --> Walk{Can victim walk?}
    Walk -->|Yes| GREEN[GREEN - Minor]
    Walk -->|No| Breath{Breathing?}
    Breath -->|No| Airway{Breathing after airway reposition?}
    Airway -->|No| BLACK[BLACK - Deceased]
    Airway -->|Yes| RR{Respiratory rate > 30?}
    Breath -->|Yes| RR
    RR -->|Yes| RED[RED - Immediate]
    RR -->|No| Pulse{Radial pulse OK / cap refill < 2s?}
    Pulse -->|No| RED
    Pulse -->|Yes| Mental{Follows simple commands?}
    Mental -->|No| RED
    Mental -->|Yes| YELLOW[YELLOW - Delayed]
    GREEN --> Route[Trauma-Tier Routing]
    YELLOW --> Route
    RED --> Route
    BLACK --> Police[Police / 108 notify]
    Route --> End([End])
```

GHP[Build GHP + QR] `

10. START Triage FSM Specification

States

State	Purpose
AMBULATORY 	Can victim walk?
BREATHING_CHECK 	Are they breathing?
AIRWAY_REPOSITION 	After airway maneuver, breathing?
RESPIRATORY_RATE 	RR > 30/min?
PERFUSION_CHECK 	Radial pulse / capillary refill
MENTAL_STATUS 	Follows simple commands?
TAGGED 	Final triage color assigned

Correct Transition Rules (Critical – Do Not Simplify Wrong)

Condition	Result
Can walk	GREEN → done
Not breathing → airway fails	BLACK → done
Not breathing → airway succeeds	→ RESPIRATORY_RATE (NOT immediate RED)
RR > 30	RED
No radial pulse OR cap refill ≥ 2s	RED
Cannot follow commands	RED
All checks pass, non-ambulatory	YELLOW

FSM Interface (TypeScript)

```
`typescript export type TriageState = | 'AMBULATORY' | 'BREATHING_CHECK' | 'AIRWAY_REPOSITION'
```

'RESPIRATORY_RATE'	'PERFUSION_CHECK'	'MENTAL_STATUS'	'TAGGED';
--------------------	-------------------	-----------------	-----------

```
export interface TriageContext { count: number; canWalk: boolean; breathing: boolean;
respiratoryRateOver30: boolean; capillaryRefillOk: boolean; followsCommands: boolean;
severeBleeding: boolean; triageResult: 'RED' | 'YELLOW' | 'GREEN' | 'BLACK'; stateLog:
string[]; }`
```

Medical Disclaimer (Required in UI)

RelaySaathi provides decision support only. It is not a medical diagnosis. In an emergency, always call 108/112 when possible.

11. Golden Hour Packet (GHP) Specification

Schema

```
`typescript export interface GoldenHourPacket { id: string; // UUID v4 createdAt: string; //
ISO8601 triage: 'RED' | 'YELLOW' | 'GREEN' | 'BLACK'; location: { lat: number; // 6 decimal
places lng: number; landmark?: string; nhCode?: string; // e.g. "NH48" nhKm?: number; };
victims: { count: number; canWalk: boolean; breathing: boolean; severeBleeding: boolean;
capillaryRefillOk: boolean; followsCommands: boolean; }; routing: { facilityName: string;
facilityType: 'trauma' | 'hospital' | 'clinic'; phone: string; etaMinutes: number;
distanceKm: number; }; emergency: { dial: string; // "108" | "112" | "102" state: string;
language: 'en' | 'hi' | 'ta'; }; relayChain: Array<{ at: string; deviceHint: string }>;
integrity: string; // SHA-256 hex digest }`
```

Encoding Strategy

Step	Method
1	Build minimal JSON or pipe-delimited string
2	Compress with lz-string (compressToEncodedURIComponent)

Step	Method
3	If payload > 800 bytes → QR carries only { id, lat, lng, triage, integrity } ; full GHP on screen + IndexedDB
4	Integrity via Web Crypto SHA-256 (not HMAC – no client secrets)

SMS Template Example (English)

🚒 ROAD EMERGENCY - RelaySaathi Triage: RED | Victims: 1 Location: NH48 km 87 (13.082700, 80.270700) Injuries: Not walking, breathing, severe bleeding suspected TAKE TO: Apollo Trauma Center (12 min / 8.4 km) Call: 108 (Tamil Nadu) Generated offline via bystander relay. `

12. Data Pipeline & Database

SQLite Schema

```
`sql CREATE TABLE emergency_nodes ( id INTEGER PRIMARY KEY, -- OSM node id name TEXT NOT NULL, lat REAL NOT NULL, lng REAL NOT NULL, type TEXT NOT NULL, -- hospital, clinic, police, ambulance trauma_tier INTEGER NOT NULL, -- 1=trauma center, 2=hospital ER, 3=clinic emergency_phone TEXT, nh_corridor TEXT, state TEXT, verified INTEGER DEFAULT 0, -- 1 if manually verified verified_at TEXT ); CREATE INDEX idx_coords ON emergency_nodes(lat, lng); CREATE INDEX idx_tier ON emergency_nodes(truma_tier); `
```

trauma_tier Assignment Heuristics

Tier	Signals in OSM name/tags
1	"trauma", "AIIMS", "medical college", "super specialty"
2	"district hospital", "general hospital", healthcare=hospital
3	clinic, PHC, small dispensary

Important: Manually verify top 50 POIs in demo corridor. Do **not** default all missing phones to

"108" — mark as unknown and exclude from "verified count" claims.

Demo Corridors (Pick One Primary)

Corridor	Bbox (south,west,north,east)	Notes
Chennai-Bengaluru NH48	12.5,77.5,13.2,80.3 (expand as needed)	Near IIT Madras relevance
Delhi-Jaipur NH48	26.5,75.5,28.7,77.3	High traffic
Kathmandu valley (BIMSTEC)	Separate ingest pass	International track appeal

Overpass Query Pattern (Use nwr, not nodes only)

```
`overpass [out:json][timeout:120]; ( nwr"amenity"="hospital"; nwr"amenity"="clinic"; nwr"emergency"="ambulance_station"; nwr"amenity"="police"; ); out center; `
```

13. Offline & Spatial Routing

Spatial Query Algorithm

```
`  
User GPS (lat, lng) + triage color  
Compute bbox delta (~50 km radius)  
SQL: SELECT FROM emergency_nodes WHERE lat/lng in bbox AND trauma_tier matches triage  
For each result: Haversine distance  
Sort ascending by distance  
Return top N (display top 3 on map)
```

ETA Calculation

Mode	Formula
Offline fallback	<code>etaMinutes = (distanceKm / 40) 60</code> (assume 40 km/h ambulance avg)
Online enhancement	<code>OSRM /route/v1/driving/{lng,lat};{lng,lat}</code> → duration

Document the 40 km/h assumption in the Word doc submission.

sql.js Critical Rule

``typescript // ✔ CORRECT – bundled locally locateFile: () ⇒ '/sql-wasm.wasm' // ✖ WRONG – breaks offline demo locateFile: (file) ⇒ https://sql.js.org/dist/${file} ``

14. AI Chatbot Architecture

Hybrid Model: LLM-as-Skin, FSM-as-Spine

`` User natural language ↓ ┌──────────┐ | ONLINE | OFFLINE | LLM extracts | Regex / keyword patterns | structured slots | OR scripted button options ┌──────────┐ ↓ START Triage FSM (always runs) ↓ Deterministic triage color ``

LLM Policy

Requirement	Choice
Hackathon open-source preference	Primary: Llama 3 / Mistral via Groq or Ollama
Offline	FSM + scripted prompts – never depend on LLM for demo

Requirement	Choice
Role	Slot filling only – FSM makes all medical/routing decisions

Example Slot Filling

User says	Extracted slot
"He can't walk"	canWalk: false
"Breathing very fast"	respiratoryRateOver30: true
"Lots of blood"	severeBleeding: true
"NH48 near km 87"	nhCode: 'NH48', nhKm: 87

Voice Input

- **Web Speech API** – triggered by mic button (user gesture required)
- Support **English, Hindi, Tamil** responses minimum

15. Human QR Relay System

QR Payload Strategy

Payload size	QR contains	Also show on screen
Small (<800 bytes)	Full compressed GHP	Human-readable brief
Large (>800 bytes)	{ id, triage, lat, lng, integrity }	Full GHP (bystander reads + verifies)

Bystander Relay Logic

```
`typescript // Pseudocode onQRScanned(payload) { const ghp = deserializeAndVerify(payload); if
```

```
(!ghp) return showError('Tampered or corrupted packet'); ghp.relayChain.push({ at: new Date().toISOString(), deviceHint: getDeviceId() }); await idb.put('relay_packets', ghp); if (navigator.onLine) dispatchSMS(ghp); else window.addEventListener('online', () => dispatchSMS(ghp), { once: true }); }`
```

iOS PWA SMS Fallback

sms: URIs may fail in installed iOS PWA mode. Fallback chain:

```
navigator.share({ text: ghpSmsText })
```

Copy to clipboard + toast notification

Display full text for manual send

16. Tech Stack

Layer	Technology	Why
Frontend	Next.js 14 (App Router)	React PWA, SSR optional, team familiarity
Styling	Tailwind CSS	Fast UI
PWA / Offline	Servist or @ducanh2912/next-pwa	Service Worker caching
Maps	Leaflet + OpenStreetMap tiles	Free, cacheable tiles
Client DB	sql.js + emergency_seed.db	Offline SQL queries in browser
Client storage	IndexedDB (via idb library)	Relay packet persistence
QR	qrcode + html5-qrcode or @yudiel/react-qr-scanner	Generate + scan
Compression	lz-string	QR size reduction
Hashing	Web Crypto API (SHA-256)	Integrity check
Routing (online)	OSRM public or self-hosted	Road ETA
LLM (online)	Groq (Llama 3) or Ollama local	Open-source preference
Backend (optional)	FastAPI (Python)	POI ingest helpers, optional GHP audit POST
Data ingest	Python 3 + Overpass API	Build-time pipeline
Tests	Vitest or Jest	FSM + encoder unit tests

17. Project Structure

~ roadsafetyhackathon/ |— docs/ | — RELAYSATHI_MASTER_BRIEF.md ← this document |— scripts/

```

| └─ ingestCorridors.py # Overpass → SQLite | └─ verify_pois.py # Manual verification helper
└─ public/ | └─ emergency_seed.db # Compiled POI database | └─ sql-wasm.wasm # Bundled
locally (CRITICAL) | └─ manifest.json # PWA manifest | └─ icons/ # PWA icons | └─ src/ | └─
app/ | | └─ page.tsx # Landing gate (3 buttons) | | └─ triage/page.tsx # Chat + FSM + GHP +
QR | | └─ scan/page.tsx # Bystander QR scanner | | └─ dashboard/page.tsx # Offline map view
| | └─ api/ | | └─ chat/route.ts # LLM slot-filling proxy (optional) | └─ db/ | | └─
sqliteClient.ts # sql.js init + spatial queries | └─ utils/ | | └─ startTriageFSM.ts # START
state machine | | └─ ghpCompressor.ts # Serialize, lz-string, hash | | └─ offlineRouter.ts #
Haversine + tier filter + ETA | | └─ smsFormatter.ts # 108 SMS templates (en/hi/ta) | | └─
llmSlotFiller.ts # Online NL → slots | └─ components/ | | └─ ChatBox.tsx | | └─
QRGenerator.tsx | | └─ QRScanner.tsx | | └─ OfflineMap.tsx | | └─ TriageSummary.tsx | └─
lib/ | └─ idb.ts # IndexedDB wrapper | └─ tests/ | └─ startTriageFSM.test.ts | └─
ghpCompressor.test.ts | └─ offlineRouter.test.ts | └─ submission/ | └─ slides/ # Exactly 7
slides | └─ documentation.docx # Dependencies + assumptions | └─ package.json | └─
next.config.js | └─ README.md `

```

18. Implementation Plan (4 Weeks)

Week 1 — Data Foundation

Day	Task	Owner	Done?
1-2	Finalize demo corridor bbox	Team lead	[]
2-3	Write + run <code>ingestCorridors.py</code> with nwr queries	Data eng	[]
3-4	Manual verify top 50 POIs (spreadsheet)	All	[]
4-5	Bundle <code>emergency_seed.db</code> + <code>sql-wasm.wasm</code> in <code>/public</code>	Backend	[]
5	Implement <code>sqliteClient.ts</code> + basic spatial query test	Frontend	[]

Week 2 – Core Logic

Day	Task	Owner	Done?
1-2	Implement <code>startTriageFSM.ts</code> with correct airway transitions	Frontend	[]
2-3	Implement <code>offlineRouter.ts</code> (Haversine + tier filter + ETA)	Frontend	[]
3-4	Implement <code>ghpCompressor.ts</code> (lz-string + SHA-256)	Frontend	[]
4-5	Unit tests for FSM, encoder, router	QA/Dev	[]

Week 3 – UI & Integration

Day	Task	Owner	Done?
1-2	Landing gate + triage chat UI	Frontend	[]
2-3	GHP summary screen + QR generator	Frontend	[]
3-4	Bystander scan page + IndexedDB relay	Frontend	[]
4-5	SMS formatter + online event trigger	Frontend	[]
5	Serwist offline caching configured	Frontend	[]

Week 4 – Polish & Submit

Day	Task	Owner	Done?
1	LLM integration (Groq/Ollama) for online slot filling	AI/Dev	[]
2	Hindi + Tamil UI strings + SMS templates	All	[]
3	Offline demo rehearsal (airplane mode test)	All	[]
4	7 slides + Word doc + README	Presenter	[]
5	Unstop submission + backup demo video	Team lead	[]

19. Execution Checklist by Role

Frontend Developer

- ☐ Next.js PWA shell + Serwist
- ☐ Chat UI wired to FSM
- ☐ QR generate + scan flows
- ☐ Leaflet map with POI markers
- ☐ iOS SMS fallback (share / clipboard)

Data / Backend Developer

- ☐ Overpass ingest script
- ☐ trauma_tier heuristics + manual verify CSV
- ☒ emergency_seed.db export for submission
- ☐ Optional FastAPI audit endpoint

AI / Logic Developer

- ☐ START FSM (correct medical transitions)
- ☐ LLM prompt for slot filling
- ☐ Offline scripted fallback prompts
- ☐ Voice input integration

Presenter / PM

- ☐ 7-slide deck (problem → solution → demo → results)
- ☐ 90-second demo script memorized
- ☐ Word doc assumptions (ETA formula, POI sources, START simplification)
- ☐ Unstop registration + submission

20. Demo Day Script (90 Seconds)

Time	Action	Say
0:00	Enable airplane mode on Phone A	"NH48, zero signal – common on Indian highways."
0:10	Tap Start Emergency, voice/text input	"Victim: friend bleeding, can't walk, NH48 km 87."
0:25	FSM completes → RED → trauma center shown	"RED triage – routing to trauma center, not nearest clinic."
0:35	GHP displayed + QR shown	"Golden Hour Packet ready – everything 108 needs."
0:45	Phone B scans QR	"Victim phone dead – bystander carries the packet."
0:55	Disable airplane mode on Phone B	"Bystander drives 5 km – signal returns."
1:05	SMS to 108 pre-filled	"Information reaches dispatch before the ambulance is called."
1:15	Show POI count + offline architecture slide	"14,000+ POIs, fully offline triage, open-source stack."

Backup: Pre-record 2-minute demo video if live Wi-Fi fails.

21. Submission Requirements

7-Slide Deck Outline

Slide	Content
1	Welcome – Team name, RelaySaathi, RoadSoS
2	Problem – Golden hour, dead zones, wrong hospital routing
3	Solution – Offline AI triage chatbot + GHP + human relay
4	Architecture – FSM + SQLite + QR relay diagram
5	Innovation – Trauma-tier routing, NH km markers, multilingual
6	Demo results – POI count, offline test, sample GHP
7	Thank you + future work (volunteer network, OSRM live, BIMSTEC expansion)

Word Document Must Include

1. Complete codebase map
1. All npm/pip dependencies
1. Assumptions: ETA formula, START simplification, POI verification level
1. How to run ingest script + dev server
1. Offline demo instructions

22. Verification & Testing Plan

Automated Tests

```
`typescript // startTriageFSM.test.ts expect(triage({ canWalk: true })).toBe('GREEN');
expect(triage({ canWalk: false, breathing: false, airwayOk: false })).toBe('BLACK');
expect(triage({ canWalk: false, breathing: false, airwayOk: true
```

```
}).toContinueTo('RESPIRATORY_RATE'); expect(triage({ respiratoryRateOver30: true
}).toBe('RED'); // ghpCompressor.test.ts
expect(roundTrip(ghpWithTamilLandmark)).toEqual(ghpWithTamilLandmark);
expect(verifyIntegrity(tamperedPacket)).toBe(false); // offlineRouter.test.ts
expect(route('RED', chennaiCoords)[0].traumaTier).toBeLessThanOrEqual(2); `
```

Manual Tests (Required Before Submit)

Test	Pass criteria
Airplane mode triage	Full FSM + GHP without network
QR scan relay	Phone A → Phone B packet verified
SMS trigger	108 message readable in Tamil/Hindi/English
sql.js offline	DB loads with CDN disabled
POI search	Known hospital appears within 50 km of test coord
LLM offline fallback	Scripted prompts work when API key removed

23. Risks & Mitigations

<i>Risk</i>	<i>Impact</i>	<i>Mitigation</i>
<i>QR too large to scan</i>	<i>Demo fail</i>	<i>lz-string + minimal QR payload</i>
<i>sql.js CDN load</i>	<i>Offline fail</i>	<i>Bundle sql-wasm.wasm locally</i>
<i>Wrong START medical flow</i>	<i>Judge challenge</i>	<i>Use spec in Section 10 exactly</i>
<i>Fake POI phone numbers</i>	<i>Credibility loss</i>	<i>Manual verify; honest unverified counts</i>
<i>iOS SMS broken in PWA</i>	<i>Demo fail on iPhone</i>	<i>navigator.share() + clipboard fallback</i>
<i>LLM hallucination</i>	<i>Wrong triage</i>	<i>FSM validates all decisions</i>
<i>Overpass rate limit</i>	<i>Ingest fail</i>	<i>Retry + bbox chunking at build time</i>
<i>Scope creep</i>	<i>Nothing finished</i>	<i>Reject sensor features (Section 5)</i>

24. Competitive Advantages

Why RelaySaathi Beats Typical Submissions

Typical team builds	RelaySaathi builds
Map with hospital pins	START triage + trauma-tier routing
Generic chatbot	FSM-backed medical protocol + AI skin
Online-only POI lookup	Pre-baked SQLite + corridor packs
Sends GPS coordinates	Sends full 108-ready Golden Hour Packet
No offline story	QR human relay through dead zones
English only	Hindi + Tamil + BIMSTEC config ready

Pros of This System

- 1. **Reliable demo** – airplane mode + QR works every time
- 1. **Medically defensible** – START protocol, not made-up questions
- 1. **Clinically smarter routing** – trauma center for RED cases
- 1. **Offline-first engineering** – real dead-zone problem on NH highways
- 1. **Human relay innovation** – creative but honest (no fake mesh)
- 1. **Open-source stack** – aligns with CoERS evaluation preference
- 1. **Submission-complete** – includes required database artifact
- 1. **BIMSTEC-ready** – country packs expandable
- 1. **Stage 2 ready** – live demo on two phones, no special hardware
- 1. **Clear narrative** – one story judges remember: information survives when network doesn't

25. Glossary (Quick Reference)

Term	Brief definition
PWA	<i>Website that installs and works offline</i>
FSM	<i>Fixed step-by-step logic (Finite State Machine)</i>
START	<i>Standard medical triage protocol for accident scenes</i>
GHP	<i>Golden Hour Packet – structured emergency brief for 108</i>
POI	<i>Point of Interest – hospital, police, ambulance on map</i>
SQLite	<i>Single-file local database</i>
sql.js	<i>SQLite running inside the browser</i>
OSM	<i>OpenStreetMap – free map/POI data source</i>
Overpass API	<i>Server to query OpenStreetMap data</i>
trauma_tier	<i>Hospital capability level: 1=trauma center, 2=hospital, 3=clinic</i>
Haversine	<i>Formula for distance in km between GPS coordinates</i>
bbox	<i>Bounding box – rectangular geo filter for fast SQL</i>
OSRM	<i>Open Source Routing Machine – road travel time</i>
ETA	<i>Estimated time of arrival in minutes</i>
IndexedDB	<i>Browser database for relay packets</i>
Service Worker	<i>Script that caches app files for offline use</i>
QR relay	<i>Offline handoff via bystander scanning QR code</i>
Sneakernet	<i>Data carried by humans, not network</i>
LLM-as-skin	<i>AI makes conversation natural</i>

Term	Brief definition
FSM-as-spine	<i>Rules make all critical decisions</i>
lz-string	<i>Compression library for shrinking QR payloads</i>
SHA-256	<i>Hash function for packet integrity check</i>
Golden hour	<i>First ~60 minutes after trauma – critical for survival</i>
108 / 112	<i>Indian emergency ambulance / unified emergency numbers</i>
NH km marker	<i>Highway kilometer marker (e.g. NH48 km 87)</i>
BIMSTEC	<i>Bay of Bengal regional countries for international track</i>
CoERS	<i>Centre of Excellence for Road Safety, IIT Madras</i>
RoadSoS	<i>Our hackathon problem – emergency services access</i>

26. Open Decisions & Team Actions

Decision	Options	Deadline	Owner
<i>Primary demo corridor</i>	<i>Chennai-Bengaluru NH48 vs Delhi-Jaipur</i>	<i>Week 1 Day 1</i>	<i>Team lead</i>
<i>LLM provider</i>	<i>Groq (cloud) vs Ollama (local)</i>	<i>Week 3</i>	<i>AI dev</i>
<i>BIMSTEC pack</i>	<i>Add Nepal OR Bangladesh corridor</i>	<i>Week 4 (if time)</i>	<i>Data eng</i>
<i>Backend audit API</i>	<i>Include FastAPI POST or skip for MVP</i>	<i>Week 2</i>	<i>Team vote</i>
<i>Repo host</i>	<i>GitHub/GitLab for team collaboration</i>	<i>Immediately</i>	<i>All</i>

Immediate Next Steps (This Week)

1. Create GitHub repo with folder structure from Section 17

1. Register team on Unstop (if not done)

1. Run first Overpass ingest for chosen corridor

1. **Implement** `startTriageFSM.ts` **+ unit tests**

1. Schedule daily 15-min standup until submission

27. Contacts & Resources

Resource	Link
CoERS event page	https://coers.iitm.ac.in/event/national-road-safety-hackathon-2026/
Unstop (Indian track)	https://unstop.com/hackathons/road-safety-hackathon-2026-iit-madras-1680515
Hackathon email	hackathon@rbg.iitm.ac.in
Coordinator	Dr. Swathy – +91 9791628414
OpenStreetMap	https://www.openstreetmap.org
Overpass API	https://overpass-api.de
OSRM demo server	http://router.project-osrm.org
sql.js	https://sql.js.org
START triage reference	https://en.wikipedia.org/wiki/Simple_triage_and_rapid_treatment

Appendix A – Architecture Comparison (Before vs After)

```
`mermaid flowchart LR
  subgraph Before["Rejected Approach"]
    B1[Ultrasonic V2V]
    B2[BLE Mesh]
    B3[Crash Audio ML]
    B4[Curvature Engine]
    B5[Native Hybrid App]
  end
  subgraph After["RelaySaathi"]
    A1[START FSM Chatbot]
    A2[SQLite POI DB]
    A3[GHP Packet]
    A4[QR Human Relay]
    A5[Next.js PWA]
  end
  Before -.->|replaced by| After`
```

Appendix B – Full Runtime Sequence Diagram

```
`mermaid sequenceDiagram
  participant V as Victim Phone (Offline)
  participant FSM as START FSM
  participant DB as SQLite (sql.js)
  participant GHP as GHP Builder
  participant B as Bystander Phone
  participant IDB as IndexedDB
  participant SMS as 108 SMS
  V->>FSM: Start Emergency + GPS
  FSM->>FSM: Triage questions
  FSM->>DB: Query trauma-tier POIs
  DB-->>FSM: Ranked facilities
  FSM->>GHP: Build packet + hash
  GHP-->>V: Display QR + brief
  B->>V: Scan QR
  B->>IDB: Store verified GHP Note
  over B: Drives until signal returns
  B->>SMS: Auto-compose 108 message`
```

End of RelaySaathi Master Team Brief — share freely with all team members. **Document maintainer:** Update this file when scope, corridor, or demo script changes. Do not re-introduce rejected features without team vote.

RelaySaathi · CoERS Road Safety Hackathon 2026 · Share with all team members